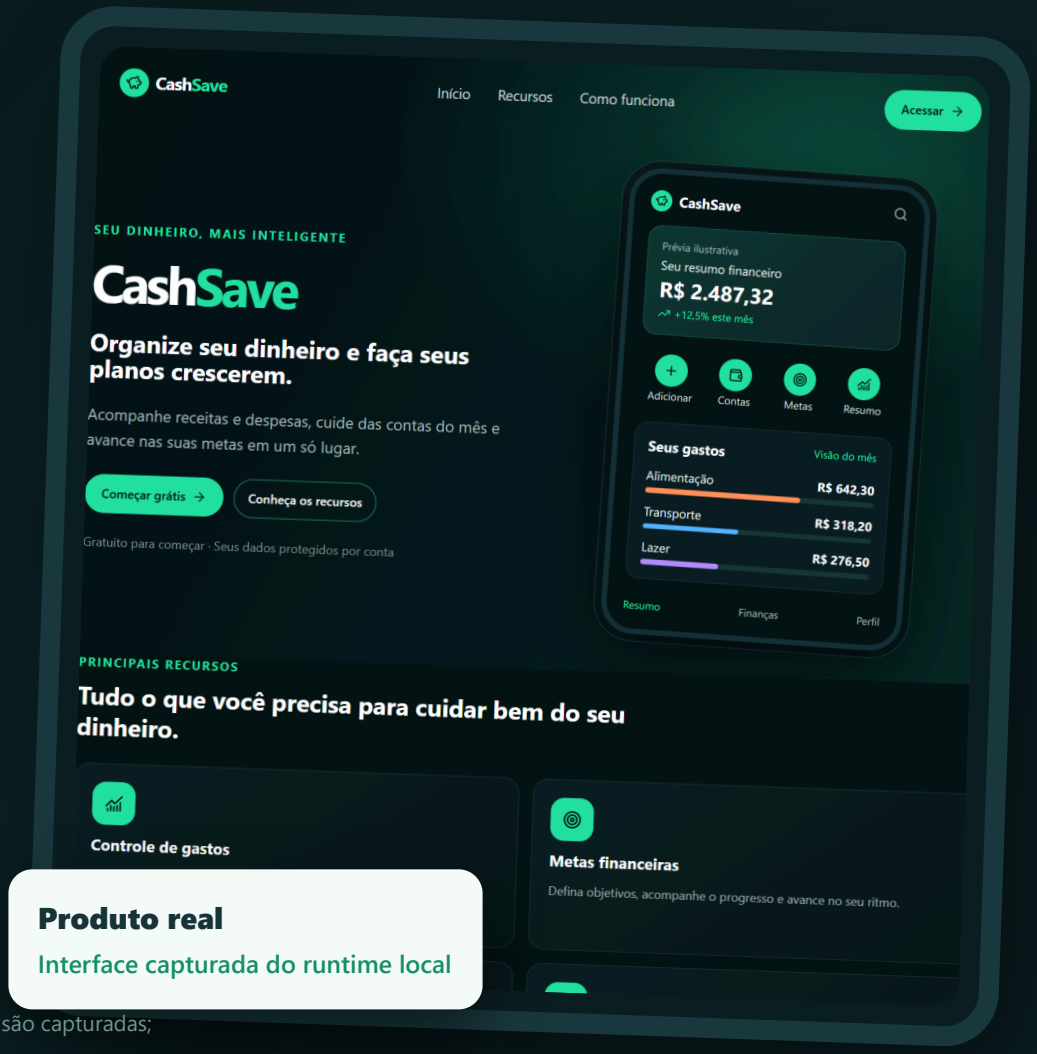
PROJETO DE DESENVOLVIMENTO

CashSave

Organização financeira com uma experiência simples, responsiva e orientada a dados.

Um aplicativo full-stack para registrar a rotina financeira, acompanhar compromissos e apoiar decisões com contexto — construído para ser claro tanto para quem usa quanto para quem mantém.

PORTFOLIO PESSOAL · 2026



Produto real
Interface capturada do runtime local

Uma leitura visual do produto, da arquitetura e das decisões de engenharia. Evidências públicas são capturadas; fluxos autenticados são descritos com seus limites reais.

O PRODUTO EM POUCOS SEGUNDOS

Menos fricção para entender o próprio dinheiro.

O CashSave transforma lançamentos, vencimentos e objetivos em uma rotina visual de acompanhamento financeiro.

Uma aplicação web de organização financeira feita em Python e Reflex, com autenticação e persistência apoiadas por APIs do Xano.

> O desafio de produto: reunir tarefas financeiras diferentes em uma interface que continue compreensível no desktop e no mobile.

O projeto atende pessoas que querem registrar receitas e despesas, acompanhar contas a pagar, planejar orçamentos e metas e consultar seus próprios dados por meio do Jarvis financeiro.

A proposta não é substituir uma decisão financeira por automação opaca: o sistema explicita limites, usa dados carregados para o usuário autenticado e mantém a IA local como recurso opcional.

11
rotas registradas no app

6
rotas públicas capturadas

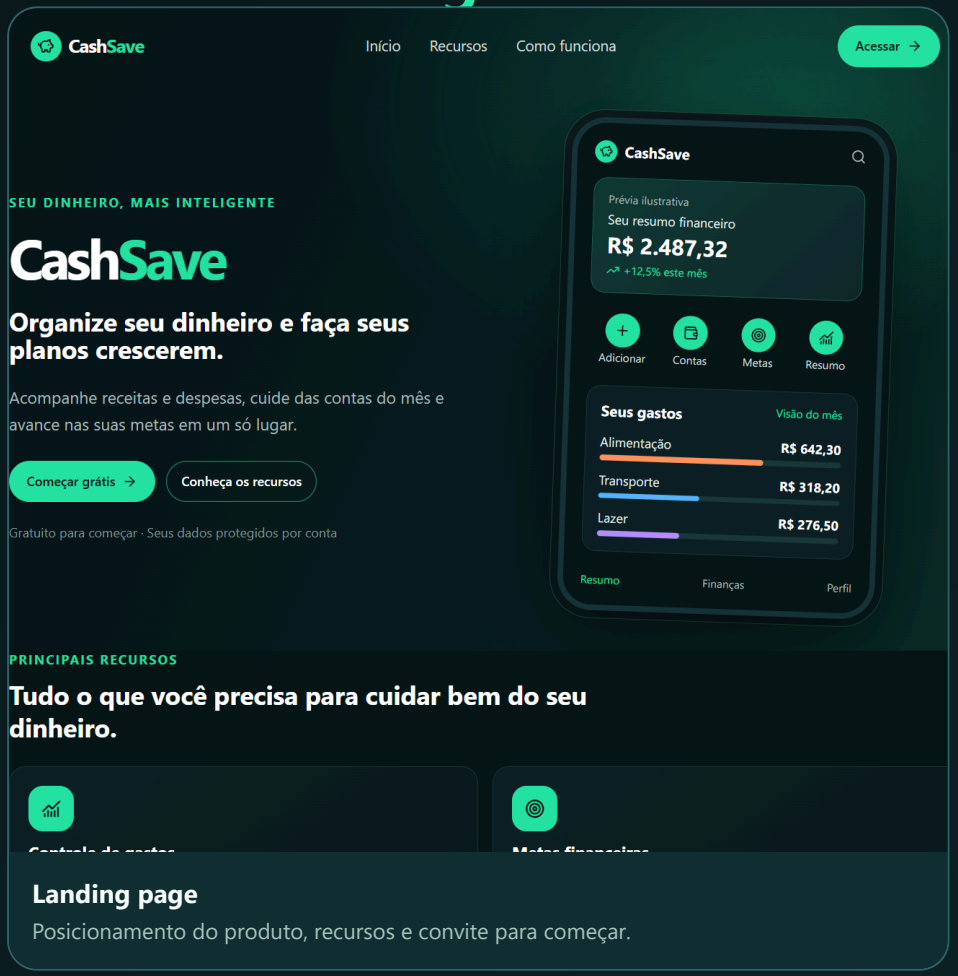
3
camadas principais: UI, estado e APIs

1
base visual coerente e responsiva

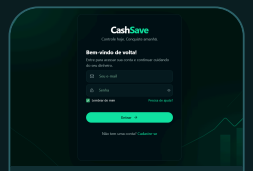
GALERIA DO RUNTIME

Uma interface que se apresenta antes mesmo do login.

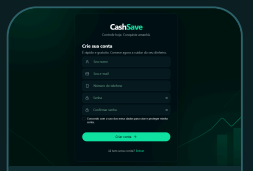
Capturas reais das rotas públicas disponíveis no ambiente local. Nenhuma tela autenticada foi simulada.



Landing page
Posicionamento do produto, recursos e convite para começar.



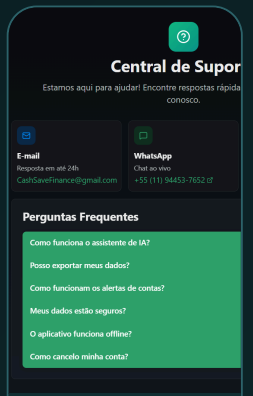
Login
Entrada com validações e sessão.



Cadastro
Criação de conta com campos estruturados.



Sobre
Contexto e proposta educativa.



Suporte
FAQ e canais de contato.

CAPACIDADES DO PRODUTO

Recursos pensados para a rotina financeira inteira.

O código mostra uma base de produto além das telas públicas. O status diferencia implementação, dependência de ambiente e evolução.

Receitas e despesas

Formulários para criar, listar e excluir transações com valor em centavos, data, tipo e descrição.

IMPLEMENTADO NO CÓDIGO

Contas a pagar

Registro de descrição, valor e vencimento, com fluxo de acompanhamento no calendário.

IMPLEMENTADO NO CÓDIGO

Orçamentos e metas

Planejamento por categoria e objetivos com progresso calculado a partir dos dados carregados.

IMPLEMENTADO NO CÓDIGO

Jarvis financeiro

Análise contextual, comandos para transações e relatório PDF de despesas; a IA não executa operações livres.

OLLAMA OPCIONAL

Calendário

Visão mensal navegável para organizar vencimentos e destacar o dia atual.

SESSÃO + XANO

Preferências

Perfil, fuso horário, notificações e avatar com upload multipart protegido.

SESSÃO + XANO

TECNOLOGIAS E HABILIDADES

Escolhas técnicas com uma função concreta.

A stack abaixo aparece diretamente nos manifests, módulos e fluxos do projeto — sem atribuir infraestrutura que não está comprovada no repositório.

Python
Linguagem da aplicação, estado, serviços e regras de domínio.

Reflex 0.9.12
Componentes reativos, páginas, eventos e compilação full-stack.

Xano APIs
Autenticação, persistência remota e operações financeiras.

ReportLab
Geração do relatório PDF de despesas no backend.

Ollama local
Integração opcional para análise financeira contextual.

python-dotenv
Configuração por ambiente sem versionar credenciais.

Habilidades demonstradas

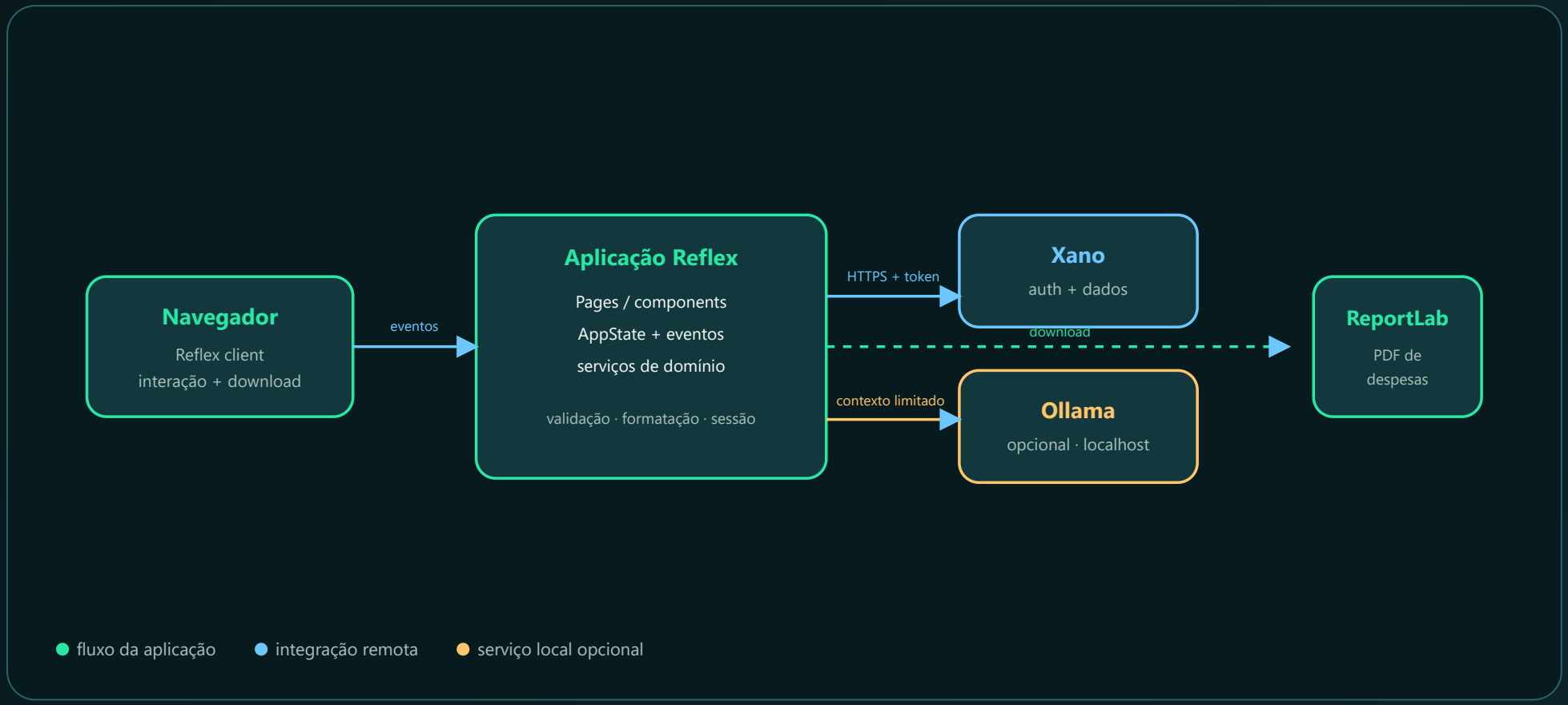
- Modelagem de estado
- Integração HTTP
- Validação de dados
- UX responsiva
- Tratamento de erros
- Testes automatizados
- Upload multipart
- Segurança de sessão
- Design de produto
- Documentação técnica

O projeto também explicita restrições: URL Xano deve usar HTTPS fora do localhost, Ollama aceita somente endpoint local e a IA recebe contexto controlado, sem SQL, código ou ferramentas livres.

COMO O SISTEMA SE ORGANIZA

Um fluxo claro entre interface, estado e serviços.

O diagrama representa as fronteiras reais observadas no código e o caminho opcional da inteligência local.



DECISÕES QUE APARECEM NO CÓDIGO

Problemas de produto tratados com limites explícitos.

Os exemplos abaixo são derivados de implementações, testes e validações presentes no repositório.

01

Dados monetários confiáveis

Solução: valores são convertidos e armazenados em centavos inteiros, com formatação brasileira e arredondamento explícito.

02

Integração remota sem falhas silenciosas

Solução: cliente Xano valida URL, timeout, respostas JSON e transforma erros HTTP em mensagens compreensíveis.

03

IA sem acesso irrestrito

Solução: Ollama é opcional, local e recebe contexto limitado; o Jarvis não executa SQL, código ou operações livres.

04

Relatório sob demanda

Solução: intenção de PDF é detectada antes do modelo e gera arquivo determinístico apenas com despesas do usuário.

05

Responsividade sem perder navegação

Solução: sidebar usa breakpoints para reduzir texto e manter ícones, enquanto conteúdo preserva largura mínima.

06

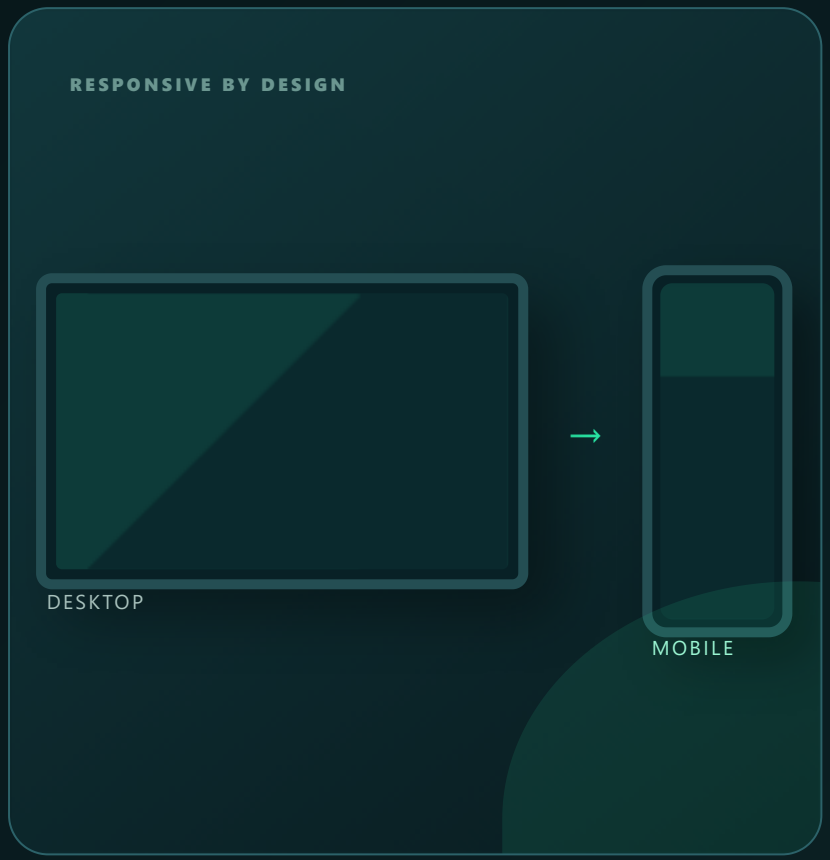
Autenticação como fronteira

Solução: rotas públicas não carregam dados autenticados; rotas protegidas têm carregamento de estado separado.

UX E COMPORTAMENTO RESPONSIVO

Consistência visual para diferentes tamanhos de tela.

O layout adapta a navegação sem esconder a lógica principal do produto: localizar, registrar e acompanhar.



01 **Hierarquia antes de decoração**
Títulos, subtítulos e cards organizam a leitura; o verde é reservado para ação, sucesso e identidade.

02 **Sidebar que se adapta**
No breakpoint inicial, a navegação reduz para 64px e preserva ícones com títulos acessíveis; em telas maiores, exibe rótulos.

03 **Feedback de estado**
O app possui estados de erro, carregamento, sessão, serviço local indisponível e confirmação de download.

04 **Dados com contexto**
Valores, datas, categorias, contas e vencimentos aparecem próximos à ação que os utiliza, reduzindo ambiguidade.

05 **Transparência operacional**
Dependências como Xano e Ollama não são mascaradas: quando indisponíveis, o produto comunica a limitação.

O QUE FOI ENTREGUE

Um projeto de produto com base para continuar evoluindo.

Resultado descrito com o mesmo critério de evidência: concluído, dependente de ambiente ou ainda candidato a evolução.

Hoje

- Aplicação Reflex estruturada em páginas, componentes e estado centralizado.
- Rotas públicas e protegidas registradas, com smoke tests de navegação.
- Integração Xano preparada para autenticação, CRUD financeiro, perfil e upload.
- Camada de análise local opcional com validação, limites e tratamento de indisponibilidade.
- Relatório de despesas em PDF gerado sem chamar o modelo.
- Testes unitários e de integração para rotas, moeda, autenticação, IA e relatórios.

Próximos objetivos

AMBIENTE DE DEMONSTRAÇÃO

Disponibilizar Xano e uma conta isolada para capturar os fluxos protegidos.

OBSERVABILIDADE

Adicionar métricas e logs estruturados para acompanhar integrações externas.

ACESSIBILIDADE

Ampliar auditoria de teclado, contraste e leitura assistiva nas telas autenticadas.

PRODUTO

Continuar refinando filtros, recorrências e visualizações sem perder simplicidade.

A principal entrega é uma experiência financeira coerente — sustentada por decisões de engenharia que deixam o sistema mais previsível, testável e honesto sobre seus limites.

SOBRE QUEM DESENVOLVEU

Conecte este projeto à sua trajetória.

Campos intencionalmente editáveis para completar antes de enviar este case study.

Identificação profissional

Nome completo

Fernando Fernandes Santos

Cargo ou área de interesse

Estudante de Análise e Desenvolvimento de Sistemas · Desenvolvimento de Software | Tecnologia da Informação

Localização / disponibilidade

São Paulo – SP

Contato e presença

GitHub

github.com/fernandofsantosdev-source

LinkedIn

linkedin.com/in/fernandofsantosdev

E-mail profissional

fernando.fsantosdev@gmail.com

Portfólio pessoal

fernandofsantosdev-source.github.io/fernando-dev-portfolio

> "Construir software é transformar uma necessidade em uma experiência que as pessoas conseguem entender, usar e confiar."

Este documento foi gerado a partir do código, dos testes e das capturas reais disponíveis no projeto CashSave.